Lumi e a Floresta Apagada

Roteiro de história + Guia de implementação de um jogo 2D em Godot 4 Versão: 1.0 | **Engine:** Godot 4.3+ (GDScript) | **Gênero:** Plataforma 2D de aventura curta | **Plataformas:** PC (Windows/Linux/Mac) e Web

> **Premissa assumida:** você não definiu gênero nem tema, então escolhi um plataforma 2D simples, com história curta e escopo realista para uma pessoa só. Se preferir top-down, puzzle ou outro tema, a estrutura do guia continua valendo e eu adapto o roteiro.

PARTE 1 – ROTEIRO DA HISTÓRIA

1.1 Logline

Um pequeno vaga-lume chamado Lumi atravessa uma floresta mergulhada na escuridão para recuperar os três fragmentos de luz do Grande Carvalho e acender a floresta de novo.

1.2 Personagens

Personagem	Papel	Traços
Lumi	Protagonista (jogável)	Vaga-lume pequeno, curioso, teimoso. Sua luz é fraca, mas cresce a cada fragmento.
Vovó Coruja	Mentora	Calma, fala por enigmas simples. Aparece no início e no final.
Sombras	Inimigos	Criaturas de névoa que fogem da luz. Não são más, só têm medo de ficar sozinhas (revelado no final).
Grande Carvalho	Objetivo	Árvore antiga que alimentava a floresta com luz.

1.3 Estrutura em três atos

Ato 1 – O Apagão (Fase 1: Raízes)

- **Cena de abertura:** a floresta está escura. Lumi acorda e vê que sua luz mal ilumina um passo à frente.
- **Diálogo (Vovó Coruja):** "O Grande Carvalho se partiu em três fragmentos de luz. Sem eles, a noite não termina."
- **Chamado à aventura:** Lumi decide buscar o primeiro fragmento, escondido nas raízes.
- **Gameplay:** tutorial de movimento e pulo, primeiras sombras, primeiro fragmento.
- **Fim do ato:** com o fragmento, a luz de Lumi cresce e revela um caminho novo.

Ato 2 – A Travessia (Fase 2: Pântano, Fase 3: Copa das Árvores)

- **Pântano:** plataformas móveis, lama que reduz o pulo, sombras mais numerosas. Lumi usa o **pulso de luz** (nova habilidade) para afastá-las.
- **Reviravolta:** Lumi encontra uma sombra encolhida, chorando. Ela não ataca. Lumi percebe que as sombras só reagem ao medo.
- **Copa das Árvores:** fase vertical, ventos, **planar** (segundo pulo curto). Segundo fragmento no topo.
- **Ponto baixo:** um ventania apaga quase toda a luz de Lumi. Vovó Coruja aparece: "A luz não é só brilhar. É não se apagar sozinho."

Ato 3 – O Despertar (Fase 4: O Grande Carvalho)

- **Clímax:** Lumi chega ao Carvalho. As sombras o cercam. Em vez de usar o pulso para afastá-las, Lumi **divide sua luz** com elas.
- **Chefe (opcional):** a Grande Sombra, que só enfraquece quando Lumi se aproxima sem atacar.
- **Resolução:** os três fragmentos se unem, o Carvalho acende e a floresta inteira brilha. As sombras viram pequenas luzes.
- **Epílogo:** Lumi no alto do Carvalho, ao lado da Vovó Coruja. Os créditos rolam sobre o amanhecer.

1.4 Tema e mensagem

Luz compartilhada vale mais que luz guardada. A mecânica final (compartilhar em vez de atacar) reforça a mensagem.

1.5 Falas principais (rascunho)

Momento	Personagem	Fala
Início	Vovó Coruja	"Acorde, pequeno. A noite está longa demais."
Fragmento 1	Lumi	"Está quente... e está vivo!"

Momento	Personagem	Fala
Sombra chorando	Lumi	"Você não quer me machucar, quer?"
Ventania	Vovó Coruja	"A luz não é só brilhar. É não se apagar sozinho."
Final	Lumi	"Não precisamos escolher entre luz e escuridão. Cabe todo mundo."

PARTE 2 – DESIGN DO JOGO

2.1 Pilares

1. **Controles justos e agradáveis** (o "game feel" vem antes de qualquer conteúdo).
2. **Luz como mecânica**: a luz de Lumi é visão, vida e arma ao mesmo tempo.
3. **Escopo curto**: 4 fases de 3 a 5 minutos cada, total de 15 a 25 minutos.

2.2 Mecânicas

Mecânica	Descrição
Movimento	Andar/voar baixo, aceleração e atrito suaves.
Pulo variável	Segurar o botão pula mais alto.
Pulso de luz	Emite onda que afasta sombras (recarga 2 s). Desbloqueado na Fase 2.
Planar	Segundo pulo curto com planeio. Desbloqueado na Fase 3.
Luz como vida	A luz diminui ao tocar sombras. Chegando a zero, volta ao último checkpoint.
Coletáveis	Pequenas centelhas aumentam a luz; fragmentos concluem a fase.
Checkpoints	Lanternas que Lumi acende.

2.3 Controles

Ação	Teclado	Gamepad
Mover	A/D ou setas	Analógico esquerdo / D-pad

Ação	Teclado	Gamepad
Pular	Espaço	A (Xbox) / Cruz
Pulso de luz	J	X / Quadrado
Pausar	Esc	Start

PARTE 3 – REQUISITOS

3.1 Requisitos Funcionais

ID	Requisito
RF-01	O jogador controla Lumi com movimento horizontal, pulo variável e, após desbloqueio, planar.
RF-02	O jogo possui menu principal, pausa, configurações (volume, tela cheia, remapear teclas) e tela de créditos.
RF-03	O jogo possui 4 fases carregadas por transição suave.
RF-04	A luz de Lumi funciona como barra de vida e raio de visão (Light2D).
RF-05	Inimigos (Sombras) causam dano por contato e reagem ao pulso de luz.
RF-06	Checkpoints salvam o ponto de respawn da fase atual.
RF-07	Diálogos aparecem em caixa de texto com avanço por botão.
RF-08	O progresso (fase atual, habilidades, coletáveis) é salvo em disco e restaurado ao abrir o jogo.
RF-09	Há música por fase e efeitos sonoros para as ações principais.
RF-10	O jogo suporta teclado e gamepad.

3.2 Requisitos Não Funcionais

ID	Requisito	Meta
RNF-01	Desempenho	60 FPS estáveis em PC modesto (4 GB de RAM, GPU integrada).
RNF-02	Tempo de carregamento entre fases	< 2 s
RNF-03	Latência de entrada	Resposta no mesmo frame (física a 60 Hz).
RNF-04	Resolução base	640x360 (pixel art) com escala inteira.
RNF-05	Tamanho do build	< 100 MB
RNF-06	Acessibilidade	Remapeamento de teclas, opção de reduzir flashes, legendas de diálogo.
RNF-07	Manutenibilidade	Cenas pequenas e reutilizáveis, scripts tipados, sinais em vez de referências diretas.

PARTE 4 – CRITÉRIOS DE ACEITE

```
CA-01 Movimento
Dado que Lumi está no chão
Quando o jogador segura "direita"
Então Lumi acelera até a velocidade máxima em até 0,15 s
E ao soltar, para em até 0,1 s

CA-02 Pulo variável
Dado que Lumi está no chão
Quando o jogador solta o botão de pulo antes de 0,2 s
Então o pulo é pelo menos 40% mais baixo que o pulo completo

CA-03 Coyote time e buffer de pulo
Dado que Lumi saiu de uma plataforma há menos de 0,1 s
Quando o jogador aperta pular
Então o pulo é executado
E se o jogador apertar pular até 0,1 s antes de pousar, o pulo ocorre ao
pousar
```

```
CA-04 Dano e respawn
Dado que a luz de Lumi chega a zero
Quando isso acontece
Então Lumi reaparece no último checkpoint em até 1 s com luz cheia

CA-05 Pulso de luz
Dado que o pulso está desbloqueado e fora da recarga
Quando o jogador aperta J
Então sombras num raio de 96 px são empurradas e ficam atordoadas por 2 s

CA-06 Salvamento
Dado que o jogador completou a Fase 2
Quando fecha e reabre o jogo e escolhe "Continuar"
Então começa na Fase 3 com o pulso de luz desbloqueado

CA-07 Desempenho
Dado qualquer fase com a quantidade máxima de inimigos e partículas
Quando o jogo roda no PC de referência
Então o FPS médio é >= 60 e nunca cai abaixo de 45

CA-08 Áudio
Quando Lumi pula, pousa, coleta e sofre dano
Então o efeito sonoro correspondente toca sem cortar a música
E os volumes de Música e Efeitos podem ser ajustados separadamente

CA-09 Entrada
Quando o jogador conecta um gamepad
Então todos os menus e o gameplay funcionam sem tocar no teclado

CA-10 Fluxo completo
Dado um novo jogo
Quando o jogador completa as 4 fases
Então vê o epílogo e os créditos e retorna ao menu principal
```

PARTE 5 – ARQUITETURA NO GODOT

5.1 Estrutura do projeto

```
lumi/
├── project.godot
├── assets/
│   ├── sprites/        (lumi/, enemies/, tiles/, ui/, fx/)
│   ├── audio/          (music/, sfx/)
│   └── fonts/
├── scenes/
│   ├── actors/         (player.tscn, shadow.tscn)
│   ├── levels/         (level_01.tscn ... level_04.tscn)
│   ├── objects/        (checkpoint.tscn, shard.tscn, spark.tscn)
│   ├── ui/             (main_menu.tscn, hud.tscn, pause.tscn, dialogue.tscn)
│   └── main.tscn
├── scripts/
│   ├── actors/
│   ├── autoload/       (game_state.gd, audio_manager.gd, scene_loader.gd,
save_system.gd)
│   ├── components/     (health_component.gd, hurtbox.gd, hitbox.gd)
│   └── ui/
├── resources/          (dados em .tres: configuração de fases, diálogos)
└── tests/
```

5.2 Autoloads (singletons)

Autoload	Responsabilidade
`GameState`	Habilidades desbloqueadas, fase atual, coletáveis. Emite sinais de mudança.
`SaveSystem`	Lê/grava `user://save.json`.
`SceneLoader`	Troca de cenas com fade e carregamento assíncrono.
`AudioManager`	Música com crossfade e pool de efeitos nos barramentos corretos.
`Events`	Barramento global de sinais (`player_died` , `shard_collected` , `dialogue_requested`).

5.3 Árvore do Player

```
Player (CharacterBody2D)
├── Sprite (AnimatedSprite2D)
├── CollisionShape2D
├── Light (PointLight2D)            # luz = vida
├── Camera2D
├── StateMachine (Node)
|   ├── Idle / Run / Jump / Fall / Glide / Hurt
├── HealthComponent (Node)
├── PulseArea (Area2D)              # alcance do pulso
└── AudioPlayers (Node)
```

5.4 Controlador do jogador (base)

```
class_name Player
extends CharacterBody2D

@export var max_speed := 140.0
@export var acceleration := 900.0
@export var friction := 1100.0
@export var jump_velocity := -320.0
@export var gravity := 900.0
@export var coyote_time := 0.1
@export var jump_buffer_time := 0.1

var _coyote := 0.0
var _buffer := 0.0

func _physics_process(delta: float) -> void:
        _apply_gravity(delta)
        _handle_horizontal(delta)
        _handle_jump(delta)
        move_and_slide()

func _apply_gravity(delta: float) -> void:
        if not is_on_floor():
                velocity.y += gravity * delta

func _handle_horizontal(delta: float) -> void:
        var dir := Input.get_axis("move_left", "move_right")
        if dir != 0.0:
                velocity.x = move_toward(velocity.x, dir * max_speed,
acceleration * delta)
        else:
                velocity.x = move_toward(velocity.x, 0.0, friction * delta)
```

```
func _handle_jump(delta: float) -> void:
        _coyote = coyote_time if is_on_floor() else _coyote - delta
        _buffer = jump_buffer_time if Input.is_action_just_pressed("jump")
else _buffer - delta

        if _buffer > 0.0 and _coyote > 0.0:
                velocity.y = jump_velocity
                _buffer = 0.0
                _coyote = 0.0
                AudioManager.play_sfx("jump")

        # Pulo variável: cortar o pulo ao soltar o botão
        if Input.is_action_just_released("jump") and velocity.y < 0.0:
                velocity.y *= 0.5
```

5.5 Sinais e desacoplamento

```
# scripts/autoload/events.gd
extends Node

signal player_died
signal shard_collected(shard_id: int)
signal light_changed(current: float, maximum: float)
signal dialogue_requested(lines: Array[String])
signal level_completed(level_id: int)
```

O HUD escuta `light_changed`; o Player apenas emite. Nenhum conhece o outro diretamente.

5.6 Gerenciador de áudio

```
# scripts/autoload/audio_manager.gd
extends Node

var _sfx: Dictionary = {
        "jump": preload("res://assets/audio/sfx/jump.wav"),
        "land": preload("res://assets/audio/sfx/land.wav"),
        "collect": preload("res://assets/audio/sfx/collect.wav"),
        "hurt": preload("res://assets/audio/sfx/hurt.wav"),
        "pulse": preload("res://assets/audio/sfx/pulse.wav"),
}

func play_sfx(name: String, pitch_variation := 0.05) -> void:
        var p := AudioStreamPlayer.new()
        p.stream = _sfx[name]
        p.bus = "SFX"
        p.pitch_scale = 1.0 + randf_range(-pitch_variation, pitch_variation)
        add_child(p)
        p.finished.connect(p.queue_free)
        p.play()
```

Crie os barramentos **Master**, **Music** e **SFX** em *Project > Audio*, para permitir volumes independentes (CA-08).

5.7 Salvamento

```
# scripts/autoload/save_system.gd
extends Node

const PATH := "user://save.json"

func save_game(data: Dictionary) -> void:
        var f := FileAccess.open(PATH, FileAccess.WRITE)
        if f:
                f.store_string(JSON.stringify(data))

func load_game() -> Dictionary:
        if not FileAccess.file_exists(PATH):
                return {}
        var f := FileAccess.open(PATH, FileAccess.READ)
        var parsed = JSON.parse_string(f.get_as_text())
        return parsed if parsed is Dictionary else {}
```

PARTE 6 – ARTE, ANIMAÇÃO E ÁUDIO

6.1 Direção de arte

- Pixel art 640x360, tiles de 16x16, sprites de 16x16 a 32x32.
- Paleta reduzida (16 a 24 cores): azuis profundos e roxos para a escuridão, amarelo/âmbar para a luz.
- Iluminação real com `PointLight2D` e `CanvasModulate` escuro por fase, que é o que faz o jogo parecer especial com arte simples.

6.2 Lista de animações

Personagem	Animações	Frames	FPS
Lumi	idle, run, jump, fall, glide, hurt, pulse	4, 6, 2, 2, 4, 2, 5	8 a 12
Sombra	idle, move, stunned, vanish	4, 4, 2, 6	8
Fragmento	pulse (loop)	6	8
Checkpoint	off, lighting, on	1, 4, 4	8

Fluxo: spritesheet PNG → `AnimatedSprite2D` com `SpriteFrames` → *Fatiar por grade*. Para animações mais complexas, use `AnimationPlayer`; para transições entre estados, `AnimationTree`.

6.3 Lista de áudio

Tipo	Itens
SFX	pulo, pouso, coleta de centelha, coleta de fragmento, dano, pulso de luz, checkpoint, clique de menu
Música	menu (calma), fases 1 e 2 (misteriosa), fase 3 (aérea), fase 4 (épica), créditos
Ambiente	grilos, vento, água do pântano (loops)

Formatos: **WAV** para efeitos curtos, **OGG** para música e ambientes.

6.4 O que eu consigo gerar para você

Sendo direto sobre o que é possível aqui:

Item	Consigo?	Como
Roteiro e diálogos	Sim	Já está neste documento.
Scripts GDScript, cenas `.tscn` e `project.godot`	Sim	Escrevo os arquivos prontos para abrir no Godot.
Sprites e spritesheets placeholder	Sim	Gero PNGs pixel art via código (Python/Pillow) ou SVG: Lumi, sombras, tiles, fragmentos. Servem para prototipar e podem ser trocados depois.
Animações	Sim, em parte	Gero os frames em spritesheet e o `SpriteFrames` configurado, além de animações procedurais por código (Tween, AnimationPlayer em `.tscn`). Não faço animação desenhada à mão com qualidade de artista.
Efeitos sonoros simples	Sim	Sintetizo WAVs por código (pulo, coleta, dano, pulso) no estilo retrô/chiptune.
Modelos 3D	Não é necessário	O jogo é 2D. Para 3D eu poderia gerar geometria básica, mas não modelos esculpidos.
Música composta e vozes	Não	Não gero áudio de música real nem dublagem. Alternativas: Sfxr/jsfxr (efeitos), LMMS ou BeepBox (música), e bancos livres como OpenGameArt, Kenney.nl e Freesound (verifique a licença).
Arte final profissional	Não	Recomendo Aseprite, LibreSprite ou Pixelorama (gratuito, feito em Godot) para a arte definitiva.

Próximo passo sugerido: me peça "gere o protótipo" e eu produzo o projeto Godot mínimo (Player + uma fase + HUD) com sprites placeholder e sons sintetizados, tudo em arquivos baixáveis.

PARTE 7 – BOAS PRÁTICAS

7.1 Game feel

- Implemente **coyote time**, **jump buffer** e **pulo variável** desde o primeiro dia.
- Adicione *juice*: partículas ao pousar, leve *screen shake* ao tomar dano, *squash and stretch* no sprite, som em toda ação.
- Use `Camera2D` com suavização e *limits* por fase.

7.2 Código

- **Tipagem estática** em todo GDScript (`var x: int`, `-> void`) para erros mais cedo e melhor desempenho.
- Siga o guia de estilo oficial: `snake_case` para variáveis e funções, `PascalCase` para classes e nós.
- **"Call down, signal up"**: pai chama filhos diretamente; filhos avisam o pai por sinais.
- Prefira **composição** (HealthComponent, Hitbox, Hurtbox) a herança profunda.
- Evite `get_node("../../X")`; use `@export var alvo: Node` ou `@onready`.
- Lógica de física em `_physics_process`, visual e entrada em `_process`.
- Use `@export` para ajustar valores no Inspector sem mexer no código; `Resource` (`.tres`) para dados como diálogos e configuração de fases.

7.3 Cenas e organização

- Uma cena por responsabilidade, pequena e reutilizável (instanciável e testável isoladamente com F6).
- Cada fase é uma cena; um `Main` troca fases via `SceneLoader`.
- Nomeie camadas de física no *Project Settings* (world, player, enemy, hazard, pickup) e use máscaras corretas.

7.4 Desempenho

- Pixel art: *Stretch Mode* `canvas_items`, *Aspect* `keep`, filtro de textura **Nearest**.
- `TileMapLayer` para cenário; evite milhares de nós individuais.
- Reutilize inimigos e projéteis com *object pooling*; evite `instantiate()` e `queue_free()` em massa por frame.
- Limite o número de `PointLight2D` ativas e o tamanho das sombras (iluminação é o item mais caro do jogo).
- Use `VisibleOnScreenNotifier2D` para pausar inimigos fora da tela.
- Perfile com o *Profiler* e *Monitors* do Godot antes de otimizar.

7.5 Controle de versão

- Use Git com o `.gitignore` oficial do Godot (ignora `.godot/`).
- Para arquivos grandes (áudio, imagens), considere Git LFS.
- Commits pequenos, um por funcionalidade; *tags* por marco (v0.1 protótipo, v0.2 fase 1...).

7.6 Entrada e acessibilidade

- Defina tudo no **Input Map** (`move_left`, `jump`, `pulse`...), nunca teclas fixas no código.
- Ofereça remapeamento, volume separado, opção de reduzir flashes e tamanho de fonte ajustável.

7.7 Testes e qualidade

- Teste manual por checklist dos critérios de aceite a cada marco.
- Para lógica pura (save, estados), use **GUT** ou **GdUnit4**.
- Teste em hardware modesto e com gamepad.
- Crie uma fase de testes ("sandbox") com todos os inimigos e objetos.

7.8 Exportação

- Instale os *export templates* da mesma versão do editor.
- Crie presets de Windows, Linux e Web; para Web, atenção a SharedArrayBuffer/CORS.
- Em release, desative o depurador e comprima texturas (VRAM Compressed para 3D; Lossless para pixel art).

7.9 Anti-padrões

- Colocar toda a lógica do jogo em um único script gigante.
- Usar `get_tree().get_nodes_in_group()` a cada frame sem necessidade.
- Valores mágicos espalhados no código em vez de `@export` ou constantes.
- Misturar interface e regra de negócio no mesmo script.
- Começar pelo conteúdo (muitas fases) antes de o movimento estar bom.

PARTE 8 – PLANO DE DESENVOLVIMENTO

Marco	Entrega	Critérios
M1 – Protótipo (semana 1)	Player com movimento, pulo variável, coyote/buffer, cenário de teste com caixas.	CA-01, CA-02, CA-03
M2 – Núcleo (semana 2)	Luz como vida, inimigo Sombra, checkpoint, respawn, HUD.	CA-04
M3 – Fase 1 completa (semana 3)	Level design, coletáveis, fragmento, diálogo inicial, arte placeholder.	RF-03, RF-07
M4 – Habilidades (semana 4)	Pulso de luz e planar, fases 2 e 3.	CA-05
M5 – Sistemas (semana 5)	Menu, pausa, opções, salvamento, gamepad.	CA-06, CA-09
M6 – Fase 4 e final (semana 6)	Chefe, epílogo, créditos.	CA-10
M7 – Polimento (semana 7)	Áudio final, partículas, balanceamento, otimização, exportação.	CA-07, CA-08

PARTE 9 – GLOSSÁRIO

- **CharacterBody2D:** nó de física para personagens controlados por código.
- **AnimatedSprite2D / SpriteFrames:** nó e recurso que tocam animações quadro a quadro.
- **Autoload:** script/cena carregado automaticamente e acessível globalmente.
- **Sinal (signal):** mecanismo de eventos do Godot para comunicação desacoplada.
- **Coyote time:** curta janela para pular depois de sair da borda.
- **Jump buffer:** registro do pulo apertado pouco antes de pousar.
- **Game feel / juice:** conjunto de pequenos efeitos que tornam o jogo satisfatório.
- **Object pooling:** reaproveitar objetos em vez de criar e destruir constantemente.

PARTE 10 – REFERÊNCIAS

- Documentação oficial do Godot 4 (docs.godotengine.org), incluindo o guia de estilo GDScript e "Best practices".
- Kenney.nl, OpenGameArt.org e itch.io (assets livres; confira as licenças).
- Ferramentas: Aseprite/LibreSprite/Pixelorama (arte), jsfxr/Sfxr (efeitos), BeepBox/LMMS (música), Tiled (opcional).
- GUT e GdUnit4 (testes em GDScript).